

Universidad de Ingeniería y Tecnología

Departamento de Ingeniería Mecatrónica



Robótica Autónoma

Profesores: Oscar E. Ramos, Alexander López

Laboratorio 2

Movimiento guiado por Sensores Exteroceptivos

Lima - Perú

2026-1

Laboratorio 2

Movimiento guiado por Sensores Exteroceptivos

Objetivos

- Detectar obstáculos que se encuentran en frente de un robot móvil y reaccionar consecuentemente usando un LiDAR.
- Detectar objetos de prueba usando OpenCV a partir de los datos de cámara publicados en un tópico de ROS.
- Mover el robot Turtlebot3 de manera reactiva usando información proveniente de la cámara y del LiDAR.

Equipo y Materiales

Descripción	Cantidad
Computadora con Internet	1

Indicaciones del Laboratorio

1. Desde el segundo laboratorio, todos los laboratorios son evaluados.
2. Se debe leer la guía del laboratorio antes de asistir a cada laboratorio.
3. El laboratorio comienza a más tardar 5 minutos después de la hora de inicio. El laboratorio empieza con la prueba de entrada, la cual se debe desarrollar de manera personal. Esta prueba de entrada cuenta como parte de la nota del laboratorio, la cual evalúa que el alumn@ haya leído la guía de laboratorio. La prueba tiene una duración de 3 minutos.
4. Los alumnos que lleguen 20 minutos después de iniciado el laboratorio son libres de asistir, pero no tendrán nota en dicho laboratorio.
5. El laboratorio se desarrolla en pareja de 2 personas (solo será de 3 personas en caso falte un alumn@ que no tenga pareja).
6. El laboratorio será evaluado durante el desarrollo del laboratorio, una vez terminada cada actividad, se llamará al profesor para evaluar dicha actividad.
7. Está prohibido copiar, en caso de copia, los alumnos involucrados serán presentados al comité de ética de la universidad.

Resumen del laboratorio

Se presenta un resumen de los puntos a tratar en el siguiente laboratorio.

1. Prueba de entrada (3 puntos).
2. Paquetes para el laboratorio.
3. Entender la transformación de sistemas de referencias (Frame Transformation)
4. Detección de obstáculos.
5. Datos de LiDAR a coordenadas cartesianas.
6. Actividades (17 puntos).

1. Paquetes del laboratorio

Para trabajar en este laboratorio, es necesario descargar el repositorio del curso en el espacio de trabajo “catkin_ws”:

```
$ git clone https://github.com/utecrobotics/robautonoma-2026 autlabs
```

Recuerde que debe tener instalado los siguientes paquetes:

```
$ sudo apt install ros-humble-gazebo-* ros-humble-cartographer ros-humble-cartographer-ros
```

```
$ sudo apt install ros-humble-navigation2 ros-humble-nav2-bringup
```

```
$ sudo apt install python3-colcon-common-extensions
```

En caso no se tenga los paquetes para trabajar con “Turtlebot3”, aquí se indica los comandos puede descargar dichos paquetes:

```
$ git clone -b humble https://github.com/ROBOTIS-GIT/DynamixelSDK.git
```

```
$ git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3.git
```

```
$ git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3_msgs.git
```

```
$ git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3_simulations.git
```

Nota: En este laboratorio se utilizará el modelo “waffle_pi” del robot “Turtlebot3”, por tanto, se debe asignar a la variable de entorno “TURTLEBOT3_MODEL” el valor de “waffle_pi”. Esta variable fue añadida al archivo “.bashrc” en el laboratorio 1, revisar el archivo para verificar que ha sido definida. En caso que se esté trabajando en “TheConstruct”, es necesario configurar en cada nuevo terminal.

2. Introducción a TF:

TF es una librería usada para trabajar y relacionar un conjunto de sistemas de referencias. Actualmente, se trabaja con la segunda llamada “TF2”, la cual permite al usuario realizar un seguimiento de múltiples sistemas de referencias a lo largo del tiempo. TF2 mantiene la relación entre los sistemas de referencias en una estructura de árbol amortiguada en el tiempo y permite al usuario transformar puntos, vectores, etc. entre dos sistemas de referencia cualesquiera en cualquier momento deseado.

2.1. Uso de TF

Un sistema robótico generalmente tiene muchos sistemas de referencias 3D que cambian con el tiempo, como un sistema de referencia del mundo, un sistema de referencia base, un sistema de referencia de agarre, un sistema de referencia de cabeza, etc. TF2 realiza un seguimiento de todos estos sistemas de referencia a lo largo del tiempo y le permite hacer preguntas como:

- ¿Dónde estaba el sistema de referencia de la cabeza en relación con el sistema de referencia del mundo hace 5 segundos?
- ¿Cuál es la pose del objeto en mi pinza en relación con mi base?
- ¿Cuál es la pose actual del sistema de referencia de la base del robot con respecto al sistema de referencia del mapa?

TF2 puede operar en un sistema distribuido. Esto significa que toda la información sobre los sistemas de referencia de un robot está disponible para todos los componentes de ROS en cualquier computadora del sistema. TF2 puede operar con un servidor central que contiene toda la información de transformación, o puede hacer que cada componente de su sistema distribuido construya su propia base de datos de información de transformación.

2.1.1. Ejemplo básico:

En un terminal ejecutar el siguiente comando para simular el robot en un mundo vacío:

```
$ ros2 launch turtlebot3_gazebo empty_world.launch.py
```

Después, ejecutar el nodo que publica el estado de las articulaciones del robot:

```
$ rviz2 # no olvide hacer el source al workspace
```

Luego teleoperar el robot a más de un metro desde donde empezó.

```
$ ros2 run turtlebot3_teleop teleop_keyboard # no olvide hacer el source al workspace
```

En el Rviz2, cambiar el “Fix Frame” a “odom” y solo agregar la opción de TF. El cuál se queda en el punto donde inició donde el robot empezó en la simulación. Se puede observar como el sistema de referencia de la odometría es el mismo que el la odometría, además de las referencia entre la ubicación de cada uno de las partes del robot con respecto del mundo. En la Figura 1 se puede observar dicho resultado:

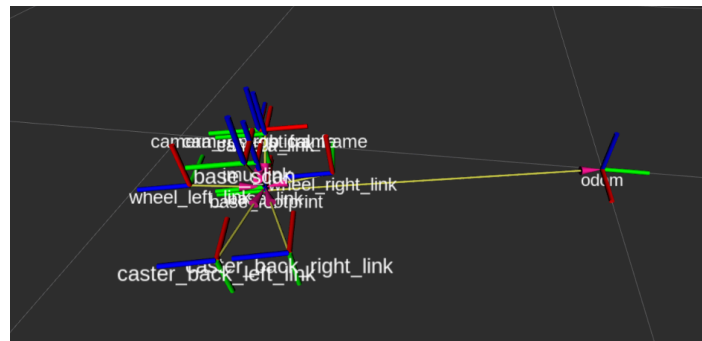


Figura 1: Ejes de transformación del robot Turtlebot3.

2.2. Nodos basados en TF

Hay esencialmente dos tareas para las que cualquier usuario usaría TF2, escuchar transformaciones y transmitir transformaciones de sistemas de referencias. Cualquiera que use TF2 deberá escuchar las transformaciones:

- Escuchar transformaciones: Reciba y almacene en un búfer todos los sistemas de referencia que se transmiten en el sistema y consulte las transformaciones específicas entre sistemas.

Para ampliar las capacidades o desarrollar un robot, deberá comenzar a transmitir transformaciones.

- Transmisión de transformaciones: Envíe la pose relativa de los sistemas de referencia al resto del sistema. Un sistema puede tener muchas emisoras, cada una de las cuales proporciona información sobre una parte diferente del robot.

2.4. Convención de nombres usados en TF:

Una parte importante del uso de TF2 es usar convenciones estándar para los sistemas de referencia. Hay varias fuentes acerca de las convenciones usadas para nombrar a los sistemas de de referencias:

- Las unidades, las convenciones de orientación, la quiralidad, las representaciones de rotación y las representaciones de covarianza se tratan en [REP 103](#).

- Los nombres estándar para sistemas de referencia de robots móviles están cubiertos en [REP 105](#).
- Los sistemas de referencia estándar para robots humanoides están en [REP 120](#).
- Para obtener definiciones de algunos de los términos matemáticos utilizados, consulte la página de [Terminología](#).

2.4.1 Denominación:

Los sistemas de referencias en ROS se identifican mediante un dato string “frame_id” en el formato de minúsculas y guiones bajos separados. Esta cadena tiene que ser única en el sistema. Todos los datos producidos pueden simplemente identificar su “frame_id” para indicar dónde se encuentra en el mundo.

Sin “tf_prefix”:

En versiones anteriores, había un concepto de tf_prefix que se anteponía al nombre del sistemas usando un separador “/”. Una barra inclinada inicial solía indicar que ya había sido prefijado. Para compatibilidad con versiones anteriores, tf2 eliminará cualquier carácter “/” inicial.

Sin reasignación:

El concepto de TF frame_ids no tiene el mismo alcance que los nombres de ROS. En particular, el espacio de nombres de una subparte específica de un gráfico de cálculo no cambia el diseño físico que representa el árbol tf. Debido a esto, frame_ids no sigue las reglas de reasignación de espacios de nombres. Es común admitir un parámetro ROS para permitir cambiar los frame_ids utilizados en los algoritmos.

Múltiples robots:

Para casos de uso con múltiples robots, generalmente se recomienda usar múltiples grupos de espacio de nombre (namespace). Puede consultar el siguiente ejemplo para tener una mejor idea de lo explicado lanzando la simulación de 3 robots en una misma simulación:

```
$ ros2 launch turtlebot3_gazebo multi_turtlebot3.launch
```

En un nuevo terminal ejecutar el siguiente comando:

```
$ sudo apt install ros-humble-rqt-tf-tree
```

```
$ ros2 run rqt_tf_tree rqt_tf_tree
```

Este comando muestra cómo cada uno de los sistemas de referencia están señalados para cada robot.

3. Detección de Obstáculos

En esta sección, se busca detectar la presencia de obstáculos utilizando la información provista por el LiDAR que se encuentra montado sobre el robot “Turtlebot3”, además de cambiar la data representada de manera a una sistema de coordenadas cartesiana. El formato de los mensajes publicados por el sensor, sea en simulación o con el robot real, no cambia; es decir, lo realizado podría ser directamente utilizado con el LiDAR que está montado sobre el robot real.

3.1. Información Cartesiana del LiDAR

El LiDAR publica en el tópico “/scan” un mensaje de tipo “LaserScan”, que se encuentra en el paquete “sensor_msgs”. Este mensaje contiene varios elementos, algunos de los más importantes son los siguientes:

- `angle_min`, `angle_max`: Indican el ángulo de inicio y fin del escaneo (en radianes).
- `angle_increment`: Indica el incremento angular entre mediciones (en radianes).
- `range_min`, `range_max`: Indica el mínimo y el máximo rango del LiDAR (en metros), tal que aquellos valores de rango que estén fuera de estos límites deben ser descartados.
- `ranges`: Es un arreglo que contiene los rangos (distancias) medidos por el LiDAR.

Para desarrollar el código, se abrirá una mundo de Gazebo que contiene al robot Turtlebot3 junto con varios objetos, usando el siguiente comando:

```
$ ros2 launch turtlebot3_gazebo turtlebot3_world.launch
```

Se debe observar un entorno similar al mostrado en la Figura 2. El lanzador carga todos los componentes necesarios del robot y de sus sensores. Para observar los frames de referencia existentes en el sistema (llamados “tf” en ROS), se puede usar el siguiente comando:

```
$ ros2 run rqt_tf_tree rqt_tf_tree
```

La información que provee `rqt_tf_tree` es solo referencial por ahora, ya que esta información se usará más adelante.

3.2. Mapa Basado en LiDAR y Odometría

La información provista por el LiDAR en la sección anterior se encuentra expresada con respecto al sistema del LiDAR, o con respecto al sistema del robot. Para construir un mapa, es necesario representarlo con respecto a un sistema de referencia inercial. En ROS, este sistema de referencia inercial se denomina “odom” (en realidad, se denomina map, pero dado que solo se utilizara la odometría en este laboratorio, se asumirá que el sistema de referencia es odom).

Al ejecutar `rqt_tree_tf`, es posible ver que el sistema del LiDAR (`base_scan`) se encuentra relacionado con el sistema de la base del robot (`base_link`), el cual a su vez se encuentra relacionado con el sistema que es su proyección sobre el piso (`base_footprint`). Todas estas transformaciones homogéneas son constantes y se publican en el tópico `tf static`. ROS tiene su propio manejo de estas transformaciones (TF). La odometría del robot, que se puede recuperar a partir del movimiento de sus ruedas, relaciona el sistema “`base_footprint`” con el sistema “`odom`”, y varía en cuanto se mueve el robot. Esta odometría se publica en el tópico “`odom`”.

4. Controlador Moteus

El controlador de motores brushless (sin escobillas) “Moteus” convierte un motor brushless en un servoactuador de alto rendimiento. Integra la electrónica de accionamiento necesaria, un microcontrolador de 32 bits de alto rendimiento, un codificador magnético absoluto y una interfaz de datos de alta velocidad en un paquete compacto. La alimentación y los datos se conectan en cadena para su integración en múltiples servosistemas. Se pueden utilizar para construir robots dinámicos.

Especificaciones:

- 3 phase brushless FOC based control
- Voltage Input: 10-44V ($\leq 10S$)
- Temperature: -40-85C
- Peak Electrical Power: 900W @ 30V
- Mass: 14.2g
- Control rate: 15-30kHz
- PWM switching rate: 15-60kHz
- 170 MHz 32 bit STM32G4 microprocessor
- Peak phase current: 100A
- Continuous phase current: 11A / 22A (w/o and w/ thermal management)
- Max electrical frequency: 3kHz
- Dimensions: 46x53mm
- Communications: 5Mbps CAN-FD
- Power Connector: 2x XT30PW-M
- Data Connector: 2x JST PH-3
- Open source firmware (Apache 2.0): <https://github.com/mjbots/moteus>
- Cada controlador también incluye 2 conectores XT30 de acoplamiento, 2 carcasas JST-PH3 de acoplamiento con terminales de crimpado y un imán magnetizado diametralmente de 1/4".

Accesorios:

- También está disponible un kit de desarrollo. Incluye un motor, el módulo mjanfd-usb-1x, la fuente de alimentación y el cableado necesario.
- Se puede usar un disipador de calor para mejorar el rendimiento térmico.
- Conexión en cadena CAN-FD: cables JST PH3 / resistencias de terminación.

La documentación está disponible en GitHub y YouTube:

[Guía de inicio](#)

[Manual de referencia](#)

[Videotutorial para usar el kit de desarrollo \(y tview\)](#)

[Videotutorial](#)

Instalar las librerías de moterus con la siguiente línea de comandos:

```
$ sudo pip3 install moteus_gui==0.3.58
```